Intelligent Reasoning Systems

Project Proposal

Project Name: RoadGuard

Group 07

YanGuanting A0305485B

HuErji A0305480L

WuTaoyi A0356972M

Contents

1. Executive Summary 1
2. Background and Problem Statement 1
    2.1 Real-world need 1
3. Business Value and Users 2
4. Objectives and Scope 3
    4.1 Objectives 3
    4.2 MVP scope 3
5. Data Plan 4
6. System Design 5
    6.1 Workflow 5
    6.2 Simplified architecture 6
7. Functional Requirements 7
    7.1 Front-end 7
    7.2 Back-end 7
8. Reasoning and Route Planning 8
    8.1 Core rules 8
    8.2 Route optimization 8
    8.3 GPT route assistant and RAG 9
9. Implementation and External APIs 9
10. Evaluation Plan 10
11. Risks, Deliverables and Conclusion 10
    11.1 Main risks 10
    11.2 Deliverables 11
    11.3 Conclusion 11

1. Executive Summary

RoadGuard is a web-based system that turns a road photo and location into a maintenance task and updated repair routes. An inspector uploads a road image and its location, then YOLO predicts the damage type, severity and confidence. If the confidence meets a configurable threshold, preset rules will automatically set the priority, create the task, add it to the queue and trigger route recalculation. Only low-confidence results wait for human review. A group-built Genetic Algorithm (GA) will use Google travel times to plan routes for two crews, while GPT with RAG system will explain the route and help users request a confirmed route change.

2. Background and Problem Statement

2.1 Real-world need

Road inspection and maintenance involve repeated observation, judgement and coordination. In practice, staff may receive many road images, but manually checking every clear result slows down task creation. Damage severity may also be judged differently by different people, and a new urgent task can quickly make an existing crew route outdated. RoadGuard connects image recognition, preset decision rules, task queues and route recalculation in one workflow. Human attention is kept for low-confidence cases, while clear cases can move forward automatically.

The project focus on five practical gaps:

Operational problem	**Effect**	**RoadGuard response**
Image review workload	Many images need the same repetitive checking	YOLO predicts damage type, severity and confidence
Uncertain predictions	Wrong detections may create wrong tasks	A confidence gate sends only low-confidence cases to human review

Operational problem	Effect	RoadGuard response
Different priority decisions	Similar cases may be treated differently	Preset rules apply the same priority mapping
New tasks make routes outdated	Crews may follow an old route plan	Queue changes automatically trigger route recalculation
Routes are hard to explain or change	Maintenance coordinator may not trust or use the route plan	GPT with RAG explains the route and converts confirmed requests into inputs for a new route calculation

3. Business Value and Users

User	Main need
Inspector / reporter	Upload an image and location
Maintenance coordinator	View the automatic queue and updated routes, ask for an explanation or request a route change,review only a low-confidence result when requested
Crew	View the maintenance queue and route map

Expected value is expressed as measurable project outcomes rather than promised financial savings:

- Clear image results can become tasks without waiting for routine manual review.
- Preset rules make severity-to-priority decisions consistent and easy to check.
- Every accepted new task updates the queue and triggers a new route calculation.
- Route planning considers urgency, travel time and crew workload.
- GPT gives a short explanation supported by the project knowledge base and handles only confirmed change requests

4. Objectives and Scope

4.1 Objectives

1. Detect the eight RDD2022ES labels that combine damage type with a basic severity distinction.
2. Use a configurable confidence threshold to separate automatic processing from human review.
3. Use preset rules to set P1/P2/P3 priority and repair advice.
4. Automatically create an accepted task, update the queue and recalculate two crew routes.
5. Use Genetic Algorithm to assign and order tasks for two crews.
6. Use GPT API with RAG system to explain the chosen route and process a user-confirmed route change.

4.2 MVP scope

In scope	**Out of scope**
Single-image upload or phone camera input	Continuous video or dashcam processing
Eight RDD2022ES damage/severity labels	Engineering-grade depth, width or structural measurement
Automatic flow above the threshold; Human review below the threshold	Unsupervised real-world deployment without logs or override
Preset priority rules and automatic queue update	Real staff, stock or contractor systems
Two-crew GA and automatic recalculation	City-wide live dispatch
GPT route explanation and a few confirmed change types	GPT directly creating an unchecked route
Responsive React web app	Native iOS or Android app

5. Data Plan

RDD2022ES is the main image dataset. It is a manually relabeled severity extension of RDD2022 with about 25,000 images, including mirrored copies, and eight detection labels: longitudinal crack / wide, transverse crack / wide, alligator crack / sunken, and pothole / deep. These labels make a basic severity-aware model easier to train than plain RDD2022, but they do not provide a complete engineering condition score. The RAG system uses a few reviewed project files containing priority rules, repair notes, route assumptions and a user guide.

Data type	**Plan**
Image model	RDD2022ES boxes with eight damage/severity-aware labels; balanced train, validation and test splits
Route test	10/15/20+ made-up tasks, one fictional department and two crews
RAG knowledge	A few reviewed files about preset rules, repair advice, route assumptions and user guide

6. System Design

6.1 Workflow

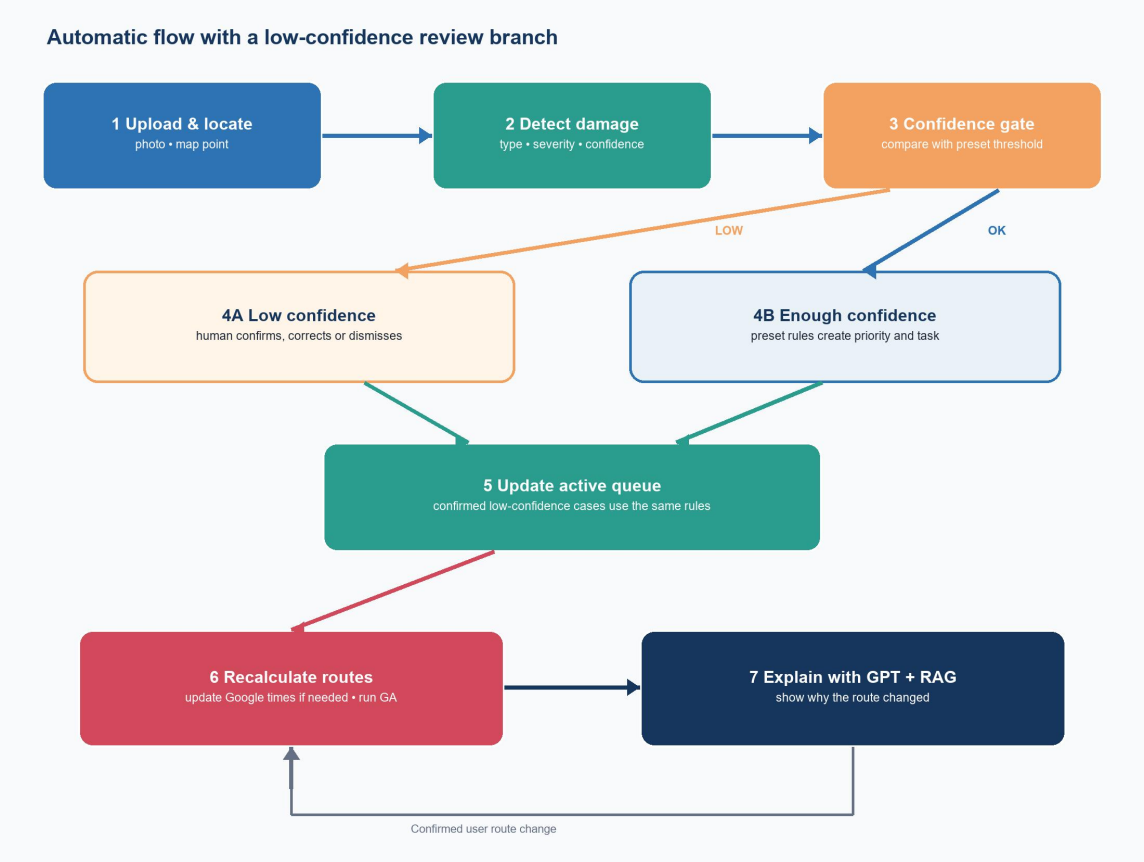


Figure 1. RoadGuard workflow

The confidence threshold is previously set and can be tuned on validation data. When confidence is high enough, the system applies the preset rules, creates the task and adds it to the active queue automatically. When confidence is too low, a staff (coordinator) confirms, corrects or dismisses the result; a confirmed result then follows the same preset rules. Every active-queue change updates any required Google travel times and triggers the GA again. GPT does not calculate the route, it explains the new result using route facts and RAG knowledge. A user-requested route change is still shown for confirmation before recalculation.

6.2 Simplified architecture

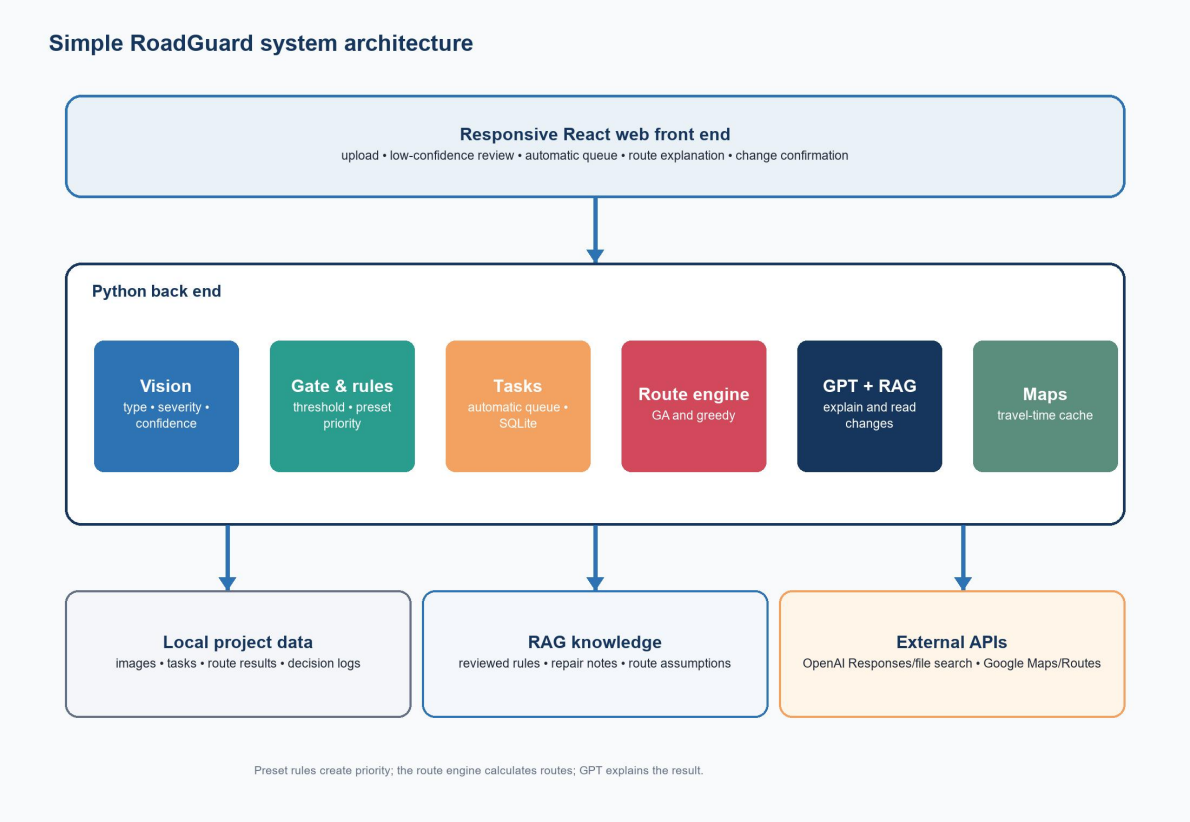


Figure 2. Simple web system

The whole system uses a React web front-end and a Python back-end. YOLO returns damage type, basic severity and confidence. A confidence gate and preset rules decide whether a case can enter the task queue automatically or must wait for human review. A queue change calls the route engine, which uses cached Google road travel times. The GPT with RAG knowledge base explain the calculated result and read supported change requests.

7. Functional Requirements

7.1 Front-end

Page	Essential functions
Inspection	Upload or take a photo, choose a location, and view damage type, severity and confidence
Review queue	Show only low-confidence cases; confirm, correct or dismiss them; priority still comes from preset rules
Tasks and routes	Show the automatically updated queue, two recalculated crew routes and route metrics
Route assistant	Ask why the route planning is like this; enter a supported change request and confirm it before route recalculation
Evaluation	View image, confidence-gate, rule, route and GPT/RAG test results

7.2 Back-end

Module	Responsibility
Vision	Run YOLO and return marked boxes, damage type, basic severity and confidence
Confidence and rules	Send low-confidence cases to human review; otherwise apply preset priority rules
Tasks	Create accepted tasks, update the active queue and save the decision trace in SQLite
Route engine	Recalculate greedy and GA routes whenever the active queue changes

Module	Responsibility
GPT and RAG	Find relevant notes, explain updated routes and read supported change requests
Maps	Get new Google travel times when needed, cache them and provide an offline backup

8. Reasoning and Route Planning

8.1 Core rules

Rule	Condition	Result
R01	Confidence < 0.60	Wait for human review; do not add an active task
R02	Confidence ≥ 0.60	Apply preset rules, create task and update the queue
R03	Wide, sunken or deep severity label	High severity; P1
R04	Base pothole or alligator-crack label	Medium severity; P2
R05	Base longitudinal or transverse crack	Low severity; P3

8.2 Route optimization

The GA creates two ordered task lists, one for each crew. It tries different task orders and keeps routes that reduce travel time, late urgent work and unfair workload. A simpler greedy method is used as a comparison. When a task enters or leaves the active queue, the route service updates the saved Google travel-time matrix for any new location and runs both methods again. Severity helps create the priority input but it will not replace the route calculation.

8.3 GPT route assistant and RAG

After each route calculation, RAG system finds useful priority rules, repair notes and route assumptions from a small reviewed knowledge base. GPT receives those notes together with the selected route, travel time, priorities and crew workload, then generates a short explanation of why routes were planned like this. It also reads a limited set of route-change requests, such as an unavailable crew, a blocked road or a new urgent task. GPT itself will never change priority or the stored route.

9. Implementation and External APIs

Area	**Choice**
Front-end	React
Image recognition	Ultralytics YOLO nano or small
Rules	Python rule engine and rules.yaml
Route planning	Python GA and simple greedy comparison
Storage	SQLite and local image folders
Google	Maps JavaScript API and Routes API route matrix
GPT	OpenAI Responses API
RAG	ChromaDB with a few approved documents

10. Evaluation Plan

Area	Experiment	Metrics
Image model	Held-out RDD2022 images	Per-class Precision, Recall, F1 and mAP@0.5
Confidence gate	Automatic and review cases at several thresholds	Auto-accept precision, review rate and missed-error examples
Priority rules	20–30 normal and boundary cases	Expected result, actual result and pass rate
Route planning	Add tasks to 10/15/20-task queues; GA versus greedy	Travel time, late tasks, workload balance, runtime and successful recalculation
GPT and RAG	10 route questions and 10 supported change requests	Correct source, clear explanation, correctly read change and safe refusal
Whole system	Five desktop or phone-browser scenarios	Correct confidence branch, queue update, route update and error handling

11. Risks, Deliverables and Conclusion

11.1 Main risks

Risk	Mitigation
RDD2022ES dataset does not match Singapore roads	Use a small local test, report the gap and keep low-confidence review
Confidence threshold accepts wrong results	Tune it on validation data, log automatic decisions and allow later correction

Risk	Mitigation
GPT gives an unsupported explanation	Use reviewed RAG files, fixed output fields and confirmation for changes
Google/OpenAI cost, network or privacy issue	Cache data, limit API use, minimize shared fields and keep a demo fallback

11.2 Deliverables

- Runnable React web app with a user guide.
- RDD2022ES YOLO model, confidence gate, preset rules, automatic queue, GA, greedy method, GPT prompts and RAG files.
- Test results for severity-aware detection, confidence routing, priority rules, automatic route updates and GPT/RAG behaviour.

11.3 Conclusion

RoadGuard keeps one clear workflow: detect damage type, severity and confidence; process reliable results automatically; human review only low-confidence results; apply preset priority rules; update the task queue; recalculate routes and explain the result. RDD2022ES dataset makes the severity-to-priority step more direct, while Google travel times and the group-built GA still perform the actual route calculation. GPT with RAG system makes the result easier to understand and adjust.